Test Completion Report

SE 2226 Software Quality Assurance & Testing

Project: dr.com.tr Website and Library-DB-Express Application

May 2026

Document Information

The following table outlines the general information and context of this report:

Field	Information
Course	SE 2226 Software Quality Assurance & Testing
Project	dr.com.tr Website and Library-DB-Express Application
Team Members	Mert Alp Barakazi, Levent Özada, Nazif Tosun
Report Date	May 2026
Report Basis	ISO/IEC/IEEE 29119-2 Section 7.4 Test Completion Report outline

Executive Summary

This report summarizes the testing activities completed for the SE 2226 Software Quality Assurance & Testing final project. The project covers two main test items: the **dr.com.tr** website, tested through black-box system, functional, usability, stress, and automated GUI testing; and the **Library-DB-Express** application, tested through API workflow, unit testing, coverage analysis, and advanced mutation testing.

Metric	Result
Manual test cases reviewed	65
Manual Pass / Fail / N/A	52 Pass / 10 Fail / 3 N/A
Manual pass rate excluding N/A	83.87%
Automated GUI tests (Selenium)	12 tests across 3 core suites passed
Jest lines coverage	47.91%
Jest function coverage	54.54%
Mutation score (Stryker)	**76.25%** (81 Mutants processed)
Black-box design techniques	EP, BVA, Decision Table, Stress Testing

1 Summary of Testing Performed

Testing was performed according to the planned scope of the SE 2226 project. The **dr.com.tr** website was tested as a black-box web application because the project did not include access to its source code. The **Library-DB-Express** application was tested using white-box techniques because the source code and backend logic were available.

1.1 Test Items and Scope

The following table defines the specific test items and their respective boundaries:

Test Item	Testing Performed	Scope Notes
dr.com.tr Website	System testing, functional testing, UI testing, stress testing, usability testing, and Selenium GUI automation.	Covered user-facing flows such as login, book search, cart management, and coupon application. Payment processing was excluded.
Library-DB-Express	Unit testing, coverage analysis, mutation testing.	Covered CRUD routing logic, endpoint behavior, unit-level behavior, and assertion strength validation using Sequelize Mocks and Jest Route Mocking.

1.2 dr.com.tr Website Testing Summary

System and end-to-end testing verified the main e-commerce flow: Search Book → Select Details → Add to Cart → Apply Coupon. UI and functional testing focused on cart limits and form validations. Automated GUI testing was completed using Java and Selenium WebDriver. The suite covers boundary values, equivalence classes, and end-to-end event browsing flows. A critical security/stress test was manually performed on the coupon field to evaluate frontend crash resilience.

1.3 Library-DB-Express Testing Summary

Unit tests were executed successfully with Jest. Coverage analysis showed 47.91% line and 54.54% function coverage due to the intentional exclusion of database exception catch blocks via Sequelize mocking. To validate the actual strength of these tests, we implemented a sophisticated Mocking Architecture in Jest, capturing `res.render` payload options. This architectural upgrade allowed Stryker Mutator to achieve an exceptional **76.25% mutation score**, proving our tests mathematically catch logical inversions.

1.4 Constraints

The dr.com.tr website was tested only through black-box techniques because its source code was not available. Real payment transactions were restricted. Live-site ad-modals and pop-ups required advanced automation workarounds (e.g., `JavascriptExecutor`) to prevent `ElementClickInterceptedException` errors.

2 Deviations from Planned Testing

All planned test categories were completed for the project scope. Remaining limitations are related to live-site constraints rather than missing planned test types.

Planned Activity	Status	Deviation / Impact	Treatment
Manual functional	Completed	No major deviation.	Findings included in report.
Automated functional (Selenium)	Completed	Ad-modals blocked standard UI clicks.	Implemented `JavascriptExecutor` to directly interact with DOM.
Unit testing & Coverage	Completed	No real DB connection.	Used Sequelize Mocks; uncovered lines documented as DB error states.
Mutation Analysis	Completed	Low initial score due to unverified render payload.	Upgraded Jest mocks to validate `res.render` parameters, boosting score to 76.25%.

3 Test Completion Evaluation

The project has reached final completion status for the planned academic testing scope. The criteria evaluation is summarized below:

Completion Criterion	Evaluation	Status
All core functionalities are tested.	Search, cart management, routing, and unit behavior were tested.	Met
Automated GUI tests are implemented.	Selenium suites for ECP, BVA, and E2E flows were built.	Met
Unit tests, coverage, and mutation analysis are completed.	Jest unit tests, coverage, and optimized Stryker results are available.	Met
Critical defects are reported.	Frontend crash (Unhandled Promise) was identified and documented.	Met

3.1 Overall Completion Statement

Based on the available evidence, the SE 2226 project has reached final test completion status for the planned academic scope. Manual testing, automated GUI testing, unit testing, coverage analysis, and mutation testing have all been completed successfully. The final completion decision is conditional on the documented residual risks being understood, especially considering the external and dynamic nature of the live dr.com.tr system.

4 Factors That Blocked or Limited Progress

Several external factors and constraints influenced our testing approach. The table below details these blocking factors, their effects, and the workarounds implemented:

Blocking/Limiting Factor	Effect on Testing	Solution or Current Handling
No source code access for dr.com.tr	Limited testing to black-box user-facing behavior.	Used ECP, BVA, and Decision Table techniques through the UI.
Live-site dynamic pop-ups	Blocked standard Selenium interaction commands.	Shifted from `WebElement.click()` to DOM manipulation via `JavascriptExecutor`.

5 Test Measures

5.1 Manual Test Case Measures

A total of 65 manual test cases were executed across various modules. The pass rate and failure distribution are summarized below:

Module	**Total**	**Pass**	**Fail**	**N/A**	**Pass Rate**
Authentication Tests	15	13	2	0	86.67%
Search & Filter Tests	20	16	4	0	80.00%
Cart & BVA Tests	15	14	1	0	93.33%
Stress/Coupon Tests	15	9	3	3	75.00%
Total	**65**	**52**	**10**	**3**	**83.87%**

5.2 dr.com.tr Website Findings by Area

The critical findings from the black-box execution on the production website are categorized by feature area:

Area	Key Result	Main Issues
Cart Management	Limit validated successfully.	System enforces a 50-item limit properly but lacks proactive user feedback before the final action.
Coupon Application	Extreme inputs triggered systemic failures.	Submitting 1000+ characters generated a 403 API response that crashed the frontend (Unhandled Promise Rejection).

5.3 Library-DB-Express Measures (White-Box & Mutation)

White-box testing focused on unit-level routing, coverage, and mutation. The results are strictly tied to the Jest and Stryker test frameworks:

Measure	Result	Interpretation
Unit testing	Jest unit tests passed.	Upgraded unit tests validate not only HTTP status but also deep JSON payloads within `res.render` intercepts.
Coverage analysis	47.91% Line, 54.54% Function coverage.	Missing coverage exclusively belongs to unreachable DB error handlers (`SequelizeValidationError`) bypassed by mocks.
Mutation testing	**76.25% score** (49 Killed, 12 Timeout, 19 Survived, 1 Error).	Extremely high score achieved by injecting robust test assertions on arithmetic operators (`Math.ceil`) and view options.

5.4 Black-Box Test Design Technique Coverage

Test cases were systematically designed using the following black-box techniques to ensure maximum scenario coverage with minimal redundancy:

Technique	Coverage
Equivalence Partitioning (EP)	Partitioned valid/invalid inputs for login fields.
Boundary Value Analysis (BVA)	Cart quantities evaluated at boundaries: 1, 49, 50, and 51.
Decision Table Testing	Logical matrices applied to multi-condition user scenarios.
Negative / Stress Testing	Simulated malicious payload sizes (1000+ chars) in coupon inputs.

5.5 Selenium GUI Automation Measures

The automated UI test suites were built to replicate real user journeys. The technical parameters of the automation framework are listed below:

Measure	Result
Automation Framework	Java, Selenium WebDriver, JUnit 5
Browser Targeting	Google Chrome (ChromeDriver)
Execution Strategy	Bypassed UI interceptors using `JavascriptExecutor`
Synchronization	Explicit Waits (`WebDriverWait`) used for async element rendering
Test Suites	3 Core Suites (Search, Cart Limit, Authentication)

5.6 Defect and Risk Summary

The highest severity defect discovered was the frontend's inability to handle HTTP 403 API responses during the coupon stress test, leading to an Unhandled Promise Rejection. Additionally, cart boundary limits (50 items) lack proactive UI indicators, presenting a usability risk. For the White-Box environment, branch coverage gaps were noted in database exception blocks, though mitigated heavily by the 76.25% Stryker mutation score.

6 Test Deliverables

The following documentation, scripts, and reports constitute the final deliverables submitted for the SE 2226 project evaluation:

Deliverable	Status	Content
Final Project Presentation	Completed	Slide deck detailing Hybrid Testing Strategies.
Automated Test Scripts	Completed	Selenium Java UI tests.
Coverage & Mutation Reports	Completed	Terminal outputs and optimized 76.25% Stryker reports.
Test Completion Report	Completed	This document.

7 Lessons Learned

- **UI Resilience vs. Backend Security:** A functionally secure backend (returning 403 errors) does not guarantee system stability if the frontend lacks exception handling.
- **Automation Hurdles:** Automating production sites requires adaptive strategies (like `JavascriptExecutor`) to bypass unpredictable third-party scripts and marketing modals.
- **The Power of Deep Mocking:** By refactoring our Jest tests to intercept and assert the variables passed into `res.render`, we transformed a weak test suite into an impenetrable one, boosting our mutation score from 18% to 76.25%.
- **Boundary Prioritization:** BVA proved to be the most efficient black-box technique for identifying logical limits (e.g., the 50-item cart constraint) with minimal test cases.

Appendix A: Automation & White-Box Evidence Summary

The automation suites for both Black-Box and White-Box environments were executed using the following configurations and commands:

Item	Evidence / Execution Command
Selenium Execution Command	Executed via IntelliJ IDEA JUnit Test Runner
Jest Unit Tests & Coverage	`npx jest --coverage`
Stryker Mutation Analysis	`npx stryker run`
Stryker Reporter	HTML, Clear-text, Progress
Jest Timeout Configuration	`timeoutMS: 10000`
Mocking Strategy (Jest)	`jest.mock('../models', ...)` overriding Sequelize methods

Appendix B: Recommended Follow-Up Actions

Based on the findings from our testing lifecycle, the following actions are recommended for the development team:

Priority	Recommended Action	Reason
High	Implement `catch` blocks for API requests on the frontend (specifically for the Coupon service).	Prevents the `Unhandled Promise Rejection` that causes the UI to crash when receiving 400/403 status codes.
Medium	Add proactive UI feedback for the 50-item cart limit.	Currently, users only see the limit error after attempting to add the 51st item, reducing UX quality.
Medium	Maintain `JavascriptExecutor` methods in Selenium scripts.	Live production sites frequently change their ad-overlays, standard `click()` methods will continue to fail.
Low	Expand Jest coverage into Sequelize Error validation blocks.	Currently untested database validation errors reduce branch coverage to 0%.